

MAJOR PROJECT REPORT

Department of Computer Science and Engineering

[INFORMATION REQUIRED: institution name]

TrustLens

A Semi-Automated Framework for Trustworthiness Assessment and Auditing of Hugging Face Machine Learning Models

Evidence-driven auditing of machine-learning models along the five FRIES
dimensions:

Fairness, Robustness, Integrity, Explainability and Safety

Aayushman Roll No. 231CS105

Ashutosh Kumar Roll No. 231CS113

Chetan Kumar Sah Roll No. 231CS118

Under the guidance of
[INFORMATION REQUIRED: supervisor name]

Repository: <https://github.com/Aayushman00/TrustLens>
Academic year: [INFORMATION REQUIRED: academic year]

Certificate

This is to certify that the major project report entitled “**TrustLens: A Semi-Automated Framework for Trustworthiness Assessment and Auditing of Hugging Face Machine Learning Models**” is a record of work carried out by **Aayushman (231CS105)**, **Ashutosh Kumar (231CS113)** and **Chetan Kumar Sah (231CS118)** under my supervision, in partial fulfilment of the requirements of the degree programme at [INFORMATION REQUIRED: institution name], Department of Computer Science and Engineering.

[INFORMATION REQUIRED: supervisor name]

Project Supervisor

Declaration

We declare that this report describes our own work on the TrustLens project. The software, the experimental models and the evaluation runs reported here were produced by the three of us, and every number in the results chapters is taken from result files stored in the project repository. Where we rely on other people’s ideas, datasets or models, we cite them in the reference list. An AI writing assistant was used while preparing the text of this report; the technical content was checked against the repository by the team.

Aayushman (231CS105)

Ashutosh Kumar (231CS113)

Chetan Kumar Sah
(231CS118)

Acknowledgement

We thank our supervisor, [INFORMATION REQUIRED: supervisor name], for pushing us away from a purely engineering project and towards a question we could actually test: does the tool we built behave differently on models we know to be worse? We are also grateful to the maintainers of the Hugging Face Hub, the `transformers` and `datasets` libraries and the Civil Comments dataset, whose open releases let us run all of this on a single laptop GPU. Finally, we thank our classmates, who read early drafts of the experiment design and pointed out several holes in it.

Abstract

Public model hubs make it easy to download a classifier in one line, but they give little help in deciding whether that classifier deserves trust. Model cards are free text, licences may be missing, behaviour on sensitive groups is rarely reported, and nothing stops an uploader from describing a flawed model in glowing terms. **TrustLens** is the audit framework we built to make this decision more systematic. It imports a Hugging Face model, runs five probes (one per FRIES dimension: Fairness, Robustness, Integrity, Explainability, Safety), turns the evidence into Occurrence/Severity/Detection (O/S/D) ratings, and aggregates them into a 0–10 FRIES score with a cube-root risk formula. Every probe output is stored as evidence with a SHA-256 hash.

To test whether the tool is more than a score generator, we ran a forged-model experiment. One reference model (`unitary/toxic-bert`) was compared with six *forged* toxicity classifiers that we fine-tuned or edited ourselves from `bert-base-uncased`. Each carries a flaw injected into one FRIES dimension (label-flipped identity comments, a brittle under-trained model, a blank model card, an over-claiming model card, relabelled severe-toxicity examples), and one carries two flaws. All seven models were evaluated on the same 3 000-row Civil Comments sample on the same GPU, in two complete runs.

The results are mixed, and we report them as such. Our first evaluation of the reference model turned out to be invalid: `toxic-bert` has six sigmoid outputs, our label mapping pointed at the wrong one, and the model behaved as a constant predictor. We converted it to an exactly equivalent two-label head and re-evaluated it through the same pipeline. With the corrected reference (overall FRIES 6.16), the targeted-dimension hypothesis held for five of six forged models: the label-flipped model scored lower on Fairness (6.60 against 7.32, demographic-parity gap 0.42 against 0.28 with non-overlapping intervals), the brittle model lower on Robustness (8.32 against 9.00), and the blank-card, over-claiming and compound models far lower on Explainability, Integrity and Safety. The three documentation- and metadata-flawed models (V3, V4, V6) also had clearly lower overall scores (4.08–4.76). However, the model with the injected *safety* flaw scored marginally higher on Safety than the reference (3.98 against 3.91), because the Safety probe reads the model card and never tests behaviour, and three forged models still had a higher overall score than the reference. Three of the five dimensions are rated by a language model and moved between repeated evaluations; the first-run reference score differed from the second by 1.34 points.

We conclude that TrustLens collects and organises trust-relevant evidence reliably and can separate models whose documentation and metadata are deficient, but that our experiment does not show it can establish that an arbitrary model is trustworthy.

Keywords: trustworthy AI, model auditing, FRIES score, Hugging Face, fairness, robustness, model cards, supply-chain integrity.

Contents

Certificate	ii
Declaration	iii
Acknowledgement	iv
Abstract	v
Abbreviations	xi
1 Introduction	1
1.1 Background and problem statement	1
1.2 Motivation	1
1.3 Objectives	2
1.4 Research questions	2
1.5 Scope and contributions	2
1.6 Report organisation	3
2 Literature Review and Existing Systems	4
2.1 What “trustworthy” has meant	4
2.2 Fairness, robustness and explainability measures	4
2.3 Supply-chain and provenance concerns	5
2.4 Existing systems	5
2.5 Where TrustLens sits	5
3 System Architecture	7
3.1 Implemented system versus target design	7
3.2 High-level architecture	8
3.3 Component view	8
3.4 Execution flow of an audit	9
3.5 Data flow and evaluation contract	9
3.6 Evaluation modes and assessment engines	10
4 The FRIES Framework in TrustLens	11

4.1	From evidence to score	11
4.2	Probes at a glance	12
4.3	Fairness	12
4.4	Robustness	13
4.5	Integrity	13
4.6	Explainability	13
4.7	Safety	14
4.8	O/S/D engines	14
5	Implementation Details	15
5.1	Technology stack	15
5.2	Important modules	16
5.3	Installation and execution	17
6	Experimental Design	18
6.1	Question and hypothesis	18
6.2	Models	19
6.2.1	Model A: reference model	19
6.2.2	Model B: forged variants	19
6.3	Variables and controls	21
6.4	Outcome measures	21
7	Results	23
7.1	Overall and per-dimension scores	23
7.2	Measured evidence behind the scores	25
7.3	The original reference was a constant predictor, and the corrected rerun . . .	26
7.3.1	Diagnosis	26
7.3.2	Correction	27
7.4	Run-to-run variation	27
7.5	Testing hypotheses H1 and H2	28
7.6	Cost	29
8	Discussion	30
8.1	Does TrustLens identify the trustworthiness of a model?	30
8.1.1	Evidence that supports the claim	30
8.1.2	Evidence against, and limits on the claim	30

8.1.3	Is it trustworthiness or an audit profile?	31
8.2	Questions we asked of our own design	31
8.3	Threats to validity	32
8.4	Threat model	33
8.5	Lessons from building the forged models	33
9	Testing and Performance	34
9.1	Testing strategy	34
9.2	What the tests do not cover	35
9.3	Performance and resource use	35
10	Conclusion and Future Work	36
10.1	Q1. Did we successfully implement TrustLens?	36
10.2	Q2. Did TrustLens detect measurable differences between the reference and forged models?	36
10.3	Q3. Does this prove that TrustLens can determine whether any arbitrary ML model is trustworthy?	36
10.4	Contributions, restated	37
10.5	Future work	37
	References	38
A	Repository Map	40
B	Contents of the Forged-Model Suite	41
C	Corrected-Reference Run (Run 3)	42
D	Raw Dimension Scores	43
E	Proposed O/S/D Triples	44
F	Regenerating the Report	45

List of Figures

3.1	Deployment architecture of TrustLens.	8
3.2	Main packages of the backend and their dependencies.	8
3.3	Sequence of a TrustLens evaluation.	9
3.4	Execution pipeline and the three evidence layers. Layer A outputs are never turned directly into a dimension score.	9
7.1	Overall FRIES score of every model in both runs. A vertical line joins the two runs of one model. The original reference moves the most between runs (1.34 points); the diamond is the corrected reference (Run 3).	24
7.2	Dimension scores in Run 2; the first row is the corrected reference from Run 3. The columns that are uniformly dark for the variants with filled-in cards (Explainability, Safety) and the light rows for V3, V4 and V6 show where the score differences come from.	24
7.3	Run 2 FRIES profiles of V3 (blank card) and V6 (compound) against the corrected reference R* (Run 3). R* is weakest on Integrity and Safety, but V3 and V6 fall far below it on Explainability and Safety.	25
7.4	Measured fairness and robustness evidence, Run 2. R* is the corrected reference (Run 3) and G the original one with the wrong label mapping. Left: demographic parity difference with bootstrap CI. Middle: accuracy drop under character swaps. Right: clean accuracy; the dotted line is the accuracy of a model that always answers “not toxic” on this sample.	26
7.5	Dimension scores in Run 1 (circles) and Run 2 (squares) for the three LLM-rated dimensions. Blue is the reference model; orange are the forged variants.	28

List of Tables

2.1	TrustLens compared with related work.	5
3.1	Status of architectural components, checked against the source code.	7
4.1	How each FRIES dimension is operationalised in the implemented code. . . .	12
5.1	Technology stack and role of each component.	15
5.2	Key source files and responsibilities.	16
6.1	Reference model.	19
6.2	The six forged variants. All use bert-base-uncased , seed 42, max sequence length 128. “Rows” is the training set size.	20
6.3	Experimental variables.	21
7.1	FRIES scores, Run 2. The flawed dimension of each variant is in bold. Scores are on a 0–10 scale; higher is better.	23
7.2	FRIES scores, Run 1 (V2 is the redesigned model used in both runs). . . .	23
7.3	Measured evidence, Run 2. DPD with 95% bootstrap CI; Drop is the char-swap accuracy drop in percentage points; Card cov. and Safety cov. are the fractions of required sections or checks found.	25
7.4	Corrected reference model (Run 3) against the original reference (Run 2). . .	27
7.5	Overall FRIES score by run, and the difference.	28
7.6	Hypothesis check against the corrected reference R*. “Targeted dimension” compares the variant’s score on the dimension where its flaw was injected with R*’s score on that dimension. Overall scores: R* 6.16.	29
8.1	Threats an auditing tool would like to catch, and what TrustLens does about them in its current form.	33
9.1	Groups of automated tests (names from <code>backend/tests</code>).	34
D.1	Dimension scores for both runs as stored in <code>_summary.json</code> (rounded to two decimals).	43
E.1	O/S/D triples (O,S,D) per dimension as proposed by the <code>11m_v1</code> engine, Run 2. Fairness and Robustness are heuristic bands; Integrity, Explainability and Safety are LLM-rated.	44

Abbreviations

API	Application Programming Interface
CI	Confidence interval
DPD	Demographic parity difference
EOD	Equalized-odds difference
FMEA	Failure Mode and Effects Analysis
FRIES	Fairness, Robustness, Integrity, Explainability, Safety
HF	Hugging Face
LLM	Large language model
O/S/D	Occurrence / Severity / Detection (FRIES convention: higher is better)
RBAC	Role-based access control
SHA	Secure Hash Algorithm
V1–V6	The six forged variants used in the experiment (Chapter 6)

1 Introduction

1.1 Background and problem statement

A student who needs a toxicity classifier, a sentiment model or a translation model today does not train one. They search the Hugging Face Hub, read the first few lines of the model card, check the download count and call `from_pretrained`. That workflow is fast, and it is also where trust quietly gets decided. The Hub hosts hundreds of thousands of checkpoints uploaded by anyone with an account, and an empirical study of the registry found that metadata and documentation are often incomplete or inconsistent [16]. A model card may be an untouched template. A licence field may be empty, or contradict the text of the card. A card may call a model “production ready, no known biases” while the model’s behaviour on identity-related text tells a different story.

The problem we address is therefore narrow and practical: *given a model reference and a small evaluation dataset, can a tool collect enough evidence, in a reproducible and inspectable way, to show a reviewer where the risks are?* We deliberately do not ask whether a model is “trustworthy” in an absolute sense. We do not believe any set of probes we could build in one academic year can answer that question, and the experiments in this report are designed to test the narrower claim.

1.2 Motivation

Three observations motivated the project.

1. **Trust is multi-dimensional.** Accuracy alone says nothing about fairness across groups, stability under noisy input, provenance of the weights, quality of documentation, or safety disclosures. The trustworthy-AI literature groups these concerns in several overlapping ways [5, 6].
2. **Existing scoring ideas are hard to apply.** The FRIES trust score of Rutinowski et al. [1] is attractive because it returns one 0–10 number built from an FMEA-style analysis, but the original work leaves open how measured evidence should become O/S/D ratings. A tool that applies the formula must make that mapping explicit, and must say how weak it is.
3. **Scores invite over-reading.** A single number that looks authoritative is dangerous if it is not traceable. We wanted each number in a TrustLens report to point back to stored evidence, and we wanted the report to state what the number does not mean.

1.3 Objectives

1. Build a working audit pipeline for Hugging Face text-classification models: import, local inference, five probes, scoring, review and report.
2. Keep the pipeline honest: label proxies, record skipped or insufficient evidence instead of inventing values, and stamp every evaluation with a methodology version.
3. Validate the tool experimentally by checking whether models that we deliberately made worse receive lower scores than a reference model.
4. Document, with evidence, where the tool succeeds and where it does not.

1.4 Research questions

RQ1

Does the repository implement the audit pipeline it claims to implement?

RQ2

Can TrustLens, through its FRIES evaluation, measure differences between a reference model and models with known, injected flaws?

RQ3

Does a difference between these seven models justify the claim that TrustLens can tell whether an arbitrary model is trustworthy?

RQ2 and RQ3 are separate on purpose. Passing the first is evidence about the probes; it is not evidence about the second.

1.5 Scope and contributions

In scope. Text classification models run locally with the Hugging Face `transformers` library; five FRIES probes; a heuristic and an LLM-assisted O/S/D engine; human review; JSON and PDF reports; a seven-model experiment on toxicity classification.

Out of scope. Image, audio and generative models (the robustness probe skips non-text tasks); detection of backdoors or weight tampering; certified guarantees of any kind; statistical validation against human trust judgments.

Our contributions fall into three groups, which we keep apart on purpose.

Engineering

A FastAPI/Celery/React system with a local inference backend, a content-addressed evidence store, versioned reports and 592 backend test functions (Chapter 9).

Methodological

A strict separation of measured evidence (Layer A), risk assessment (Layer B) and the FRIES score (Layer C), with explicit evidence statuses (`EVALUATED`, `PROXY`, `SKIPPED`, ...) so that missing evidence is never silently scored.

Empirical

A forged-model suite with published training manifests, and an honest comparison against a reference model, including the cases where the tool did not do what we hoped.

We do not claim a new trust metric. The FRIES formula comes from prior work; our contribution is the implementation, the evidence handling and the experiment.

1.6 Report organisation

Chapter 2 reviews related work. Chapter 3 describes the architecture as implemented, and Chapter 4 explains how each FRIES dimension is operationalised. Chapter 5 covers the technology stack and how to run the system. Chapters 6 and 7 describe and report the forged-model experiment. Chapter 8 asks whether the evidence supports the claim that TrustLens detects trustworthiness, and discusses validity and the threat model. Chapter 9 covers testing and performance, and Chapter 10 concludes.

A note on status labels. Throughout, we call a feature *implemented* only if we found it in the source code, *partial* if the code exists but covers less than the specification, and *planned* if it only appears in design documents. The project README keeps a roadmap table with the same distinction; where the README and the code differed we followed the code.

2 Literature Review and Existing Systems

2.1 What “trustworthy” has meant

Early work treated trust as a synonym for accuracy and generalisation. Later definitions broadened it. Surveys such as Kemmerzell et al. [5] and Mattioli et al. [6] collect requirements like fairness, robustness, transparency, privacy, accountability and human oversight, and then discuss how hard it is to attach a number to each. Policy documents such as the NIST AI Risk Management Framework [7] describe characteristics of trustworthy systems (valid and reliable, safe, secure, accountable and transparent, explainable, privacy-enhanced, fair) while stating that trade-offs between them need human judgment. Two points recur across these sources: no single metric captures trust, and measured properties are evidence for trust rather than trust itself. TrustLens adopts both points as design rules.

Vashistha and Farahi take a more formal route. Their I-trustworthy framework ties trust to calibration, so a model is trusted when its confidence reflects its competence [3], and their U-trustworthy work argues that expected utility, not calibration alone, is the right notion [4]. Both are about probabilistic classifiers and do not cover documentation, provenance or safety disclosure, which is where much of the practical risk on a public hub sits.

2.2 Fairness, robustness and explainability measures

Fairness. Group-fairness criteria compare a model’s behaviour across groups. Demographic parity compares positive-prediction rates; equalized odds (Hardt et al. [10]) compares true- and false-positive rates. For toxicity classification, Borkan et al. [11] showed that models trained on comments can pick up identity terms as cues and introduced the Civil Comments data with identity annotations, which we use. Mehrabi et al. [12] survey the many ways bias enters a pipeline. An important caveat for our experiment is that demographic parity does not distinguish an unfair model from a fair model facing groups with different base rates; we return to this in Chapter 8.

Robustness. Character-level perturbations are a classic cheap attack on text models. HotFlip [13] uses gradient-guided character flips, while behavioural testing frameworks such as CheckList [14] organise perturbation-style tests by capability. The TrustLens probe is much simpler than either: it replaces a few characters at random and measures the accuracy drop.

Explainability and transparency. Feature-attribution methods (SHAP, LIME) explain individual predictions. A different, documentation-centred line starts from model cards [8] and datasheets [9], which propose structured disclosure of intended use, limitations, training data and evaluation. The TrustLens Explainability and Safety probes follow this second line: they look at what a card discloses, not at model internals.

2.3 Supply-chain and provenance concerns

Pre-trained model reuse creates a software supply chain. Jiang et al. [16] describe how Hugging Face models are reused, and note risks around missing or inaccurate documentation and attribution. At the extreme, a tampered checkpoint can carry a backdoor that behaves normally on clean inputs [15]. Verifying that a downloaded file matches a trusted reference hash is the straightforward countermeasure, and the TrustLens Integrity probe implements the comparison, though in our experiment no trusted reference hash was supplied.

2.4 Existing systems

Table 2.1 compares TrustLens with the systems we studied. The comparison is based on the descriptions in the cited papers and on our own use of the Hub; we did not re-run the other tools.

Table 2.1: TrustLens compared with related work.

System	What it offers	Gap relative to our goal
TrustML [2]	Python package to define and compute composite trust indicators from a catalogue of metrics; supports monitoring.	Metric set and weights are chosen by the user; no model import, no documentation or integrity evidence.
FRIES score [1]	FMEA-inspired, five-aspect 0–10 score built from O/S/D ratings.	A methodology, not a tool; O/S/D assignment is left to experts.
I/U-trustworthy [3, 4]	Calibration- and utility-based formal definitions with statistical tests.	Focus on probabilistic classifiers; no fairness, integrity or documentation evidence.
Model cards [8]	A disclosure template.	Voluntary and unverified; nothing checks what a card says.
Hub model pages	Metadata, file list, download counts.	Descriptive only; no behavioural evidence.
TrustLens	Imports a Hub model, runs local inference probes and documentation/metadata checks, applies the FRIES formula, stores hashed evidence and supports review.	Probes are shallow (Chapter 4); the O/S/D mapping is heuristic and unvalidated.

2.5 Where TrustLens sits

We position TrustLens as an *applied integration* of known pieces: group-fairness metrics, a perturbation test, card-coverage checks and metadata checks, combined under the FRIES formula, with infrastructure for evidence storage and human review. We did not find a public tool that does exactly this for Hub models, but our search was not systematic and we make no novelty claim beyond that. The main research question the literature leaves open, and which our experiment probes in a limited way, is whether such a combined score

tracks real differences in model quality.

3 System Architecture

3.1 Implemented system versus target design

The README of the repository contains a long target architecture (model analyzer, dataset analyzer, resource checker, benchmark-first runtime estimate, user confirmation, then FRIES evaluation). When we read the source, we found that part of that diagram is implemented and part is not. We describe the *implemented* flow here and list the differences in Table 3.1, because a report that drew the target diagram as if it ran would misdescribe the project.

Table 3.1: Status of architectural components, checked against the source code.

Component	Status	Evidence in repository
Model import (meta-data only)	Implemented	<code>adapters/hf_hub.py</code> ; <code>POST /v1/models/import-hf</code> reads card, tags, licence and file list, no weights.
Local inference backend	Implemented	<code>inference/local_hf.py</code> : batched transformers inference, device selection (CUDA or CPU), softmax over logits, <code>torch.no_grad</code> .
Model inspection at draft time	Implemented	<code>inference/model_inspection.py</code> reads <code>AutoConfig</code> only (no weights).
Five FRIES probes	Implemented (shallow in places)	<code>probes/fairness.py</code> , <code>robustness.py</code> , <code>integrity.py</code> , <code>explainability.py</code> , <code>safety.py</code> .
O/S/D engines	Implemented	<code>osd/deterministic.py</code> (abstains), <code>osd/agent.py</code> (heuristic), <code>osd/hybrid.py</code> (LLM-assisted).
FRIES scorer	Implemented	<code>scoring/fries.py</code> , frozen test vectors in <code>shared/scoring/fixtures</code> .
Human review	Implemented	<code>osd/review.py</code> ; <code>AI_ASSISTED</code> mode stops at <code>AWAITING_REVIEW</code> .
Reports (JSON, PDF)	Implemented	<code>reports/builder.py</code> , <code>render.py</code> , HTML template rendered with <code>WeasyPrint</code> .
Evidence store	Implemented	SHA-256 addressed files on the local filesystem (MinIO was removed in the latest commit).
Resource checker, compatibility states	Not implemented	README roadmap phase 2 marked “absent”.
Benchmark-first runtime estimate	Not implemented	README roadmap phase 4 marked “absent”.
SHAP/LIME explanations	Not implemented	Explainability probe inspects model cards only.
Behavioural safety test set	Not implemented	Safety probe is a disclosure checklist.

3.2 High-level architecture

TrustLens is a conventional web application with an asynchronous worker. Figure 3.1 shows the deployment view, which matches `docker-compose.yml` (services `postgres`, `redis`, `api`, `worker`, `frontend`).

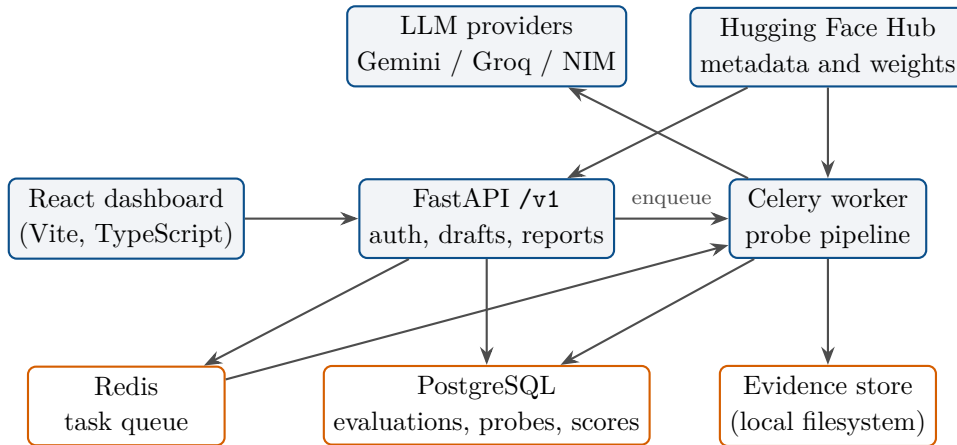


Figure 3.1: Deployment architecture of TrustLens.

The API process validates requests, stores model records and evaluation drafts, and enqueues a Celery task `trustlens.evaluate_model`. The worker loads the model through the inference backend, runs the probes, stores raw evidence, asks the O/S/D engine for ratings, calls the scorer, and writes everything back to PostgreSQL. The browser then reads the finished evaluation and its report.

3.3 Component view

Figure 3.2 groups the Python packages under `backend/app` by responsibility.

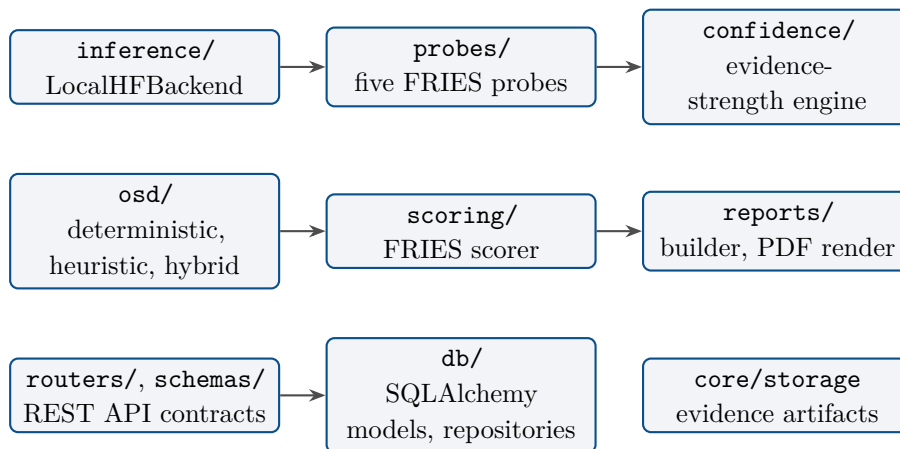


Figure 3.2: Main packages of the backend and their dependencies.

3.4 Execution flow of an audit

Figure 3.3 shows the sequence of one evaluation as the code performs it. The probe order in the runner is Fairness, Robustness, Integrity, Explainability, Safety.

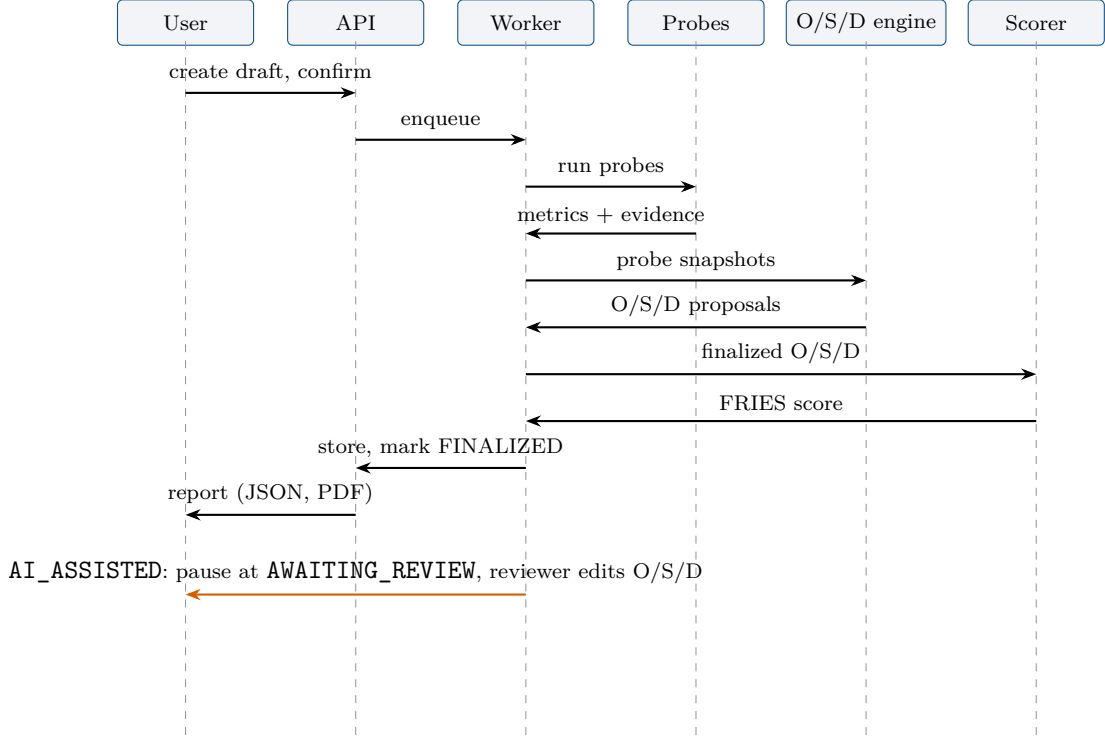


Figure 3.3: Sequence of a TrustLens evaluation.

3.5 Data flow and evaluation contract

Each evaluation carries an *evaluation contract* (schema v2 in `schemas/evaluation_contract_v2.py`) that makes the evaluation explicit rather than implicit. For Fairness and Robustness it records: the content-addressed dataset (`dataset_content_id`), the text and target columns, the sensitive-attribute column, a `label_mapping` from dataset values to model label indices, `min_group_n`, and the `positive_label_index` that demographic parity treats as the favourable outcome. It also freezes a `model_label_snapshot` (the model’s `id2label`) and the resolved model revision. We found this contract design valuable during the experiment, because it made the label mapping problem described in Chapter 7 visible after the fact: the snapshot of `unitary/toxic-bert` lists six labels, and the mapping sends dataset value 1 to index 1, which is `severe_toxic`.

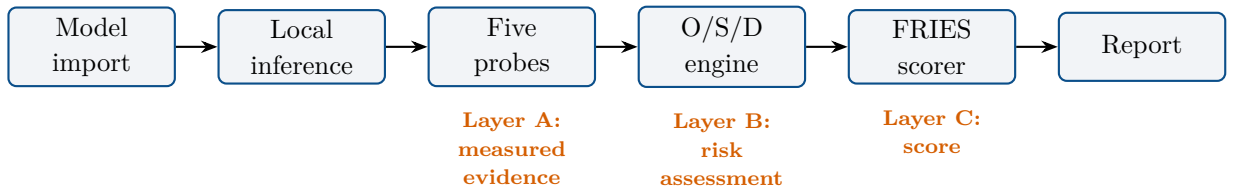


Figure 3.4: Execution pipeline and the three evidence layers. Layer A outputs are never turned directly into a dimension score.

3.6 Evaluation modes and assessment engines

Two orthogonal switches control an evaluation.

Workflow mode. `AI_AUTONOMOUS` lets the pipeline finalise without a reviewer (the report is marked `human_reviewed=false`); `AI_ASSISTED` stops so a reviewer can edit O/S/D. Despite the enum names, no language model is involved in the mode itself; they are human-review gates.

Assessment engine. `deterministic` (default) never proposes O/S/D and therefore withholds the FRIES score; `legacy_heuristic` maps probe metrics to bands with fixed rules; `llm_v1` starts from the heuristic output and then asks an LLM to re-rate Integrity, Explainability and Safety. Fairness and Robustness always keep the heuristic ratings computed from measured metrics. The experiments in this report use `llm_v1` in `AI_AUTONOMOUS` mode, which is why a repeated run can produce different numbers (Section 7.4).

4 The FRIES Framework in TrustLens

4.1 From evidence to score

FRIES, as proposed by Rutinowski et al. [1], adapts FMEA to trust. For each relevant risk an analyst rates *Occurrence* (O), *Severity* (S) and *Detection* (D), and the ratings are combined into a score. TrustLens uses the convention that **higher is better on all three scales**: a high O means the failure is unlikely, a high S means the consequence is mild, and a high D means the failure is easy to detect. Mixing this up with ordinary FMEA, where high means dangerous, would invert the result, so the scorer validates that all three values are integers in 0..10.

The arithmetic in `scoring/fries.py` is short enough to state completely:

$$\Pi_r = \sqrt[3]{O_r S_r D_r}, \quad \Pi_r = 0 \text{ if any of } O_r, S_r, D_r = 0 \text{ (veto)}, \quad (4.1)$$

$$T_a = \frac{1}{|R_a|} \sum_{r \in R_a} \Pi_r, \quad \text{FRIES} = \sum_a w_a T_a, \quad w_a = 0.2 \text{ by default.} \quad (4.2)$$

Custom weights must each be at least 0.1 and sum to one. In every experiment of this report there is one risk per dimension, so $T_a = \Pi_r$, and the weights are equal.

The geometric mean has a consequence worth stating: a single very low component pulls the risk score down much more than an arithmetic mean would. For example, $(O, S, D) = (1, 2, 2)$ gives $\Pi = \sqrt[3]{4} = 1.59$, whereas $(9, 9, 8)$ gives 8.65. This is why documentation-poor models fall so far in our results.

Three layers. TrustLens separates (A) *measured evidence* such as a parity gap or a hash, (B) *risk assessment*, the O/S/D proposal, and (C) the score. The README states that a raw metric must never be treated as a dimension score. The implemented heuristic engine does come close to violating this rule, because its bands are simple functions of one metric (for instance Fairness O is a function of the parity gap alone). The README lists this under known methodological gaps, and we repeat it here because it affects how the results should be read.

4.2 Probes at a glance

Table 4.1: How each FRIES dimension is operationalised in the implemented code.

Dimension	Input	Evidence produced	Runs the model?
Fairness	Labelled CSV with a sensitive-attribute column; label mapping	DPD, EOD, subgroup-F1 spread, bootstrap 95% CIs, group sizes	Yes (user-defined local mode); proxy logistic regression on Adult in the older mode
Robustness	Labelled text samples	Clean accuracy, accuracy after character swaps, accuracy drop with paired bootstrap CI, Wilson intervals	Yes (text classification only)
Integrity	Hub metadata of the model	Six metadata checks, named risks I-INT-*, hash gates	No (metadata); hash comparison only if two hashes are supplied
Explainability	Model card text	Presence of five required sections, contradiction flags	No (card only)
Safety	Model card text	Presence of four disclosure items, high-impact-claim flags	No (card only)

4.3 Fairness

Definition and purpose. A model is group-fair, in the sense measured here, when its behaviour does not differ much across groups defined by a sensitive attribute. It matters for toxicity moderation in particular, since an over-flagging model silences some speakers.

Implementation. `fairness_metrics.py` computes, with binary positive class $\hat{y} = \text{positive_label_index}$:

- $\text{DPD} = \max_g P(\hat{Y}=1 \mid g) - \min_g P(\hat{Y}=1 \mid g)$,
- $\text{EOD} = \max(\Delta\text{TPR}, \Delta\text{FPR})$ across groups,
- $\text{subgroup F1 spread} = \max_g F1_g - \min_g F1_g$.

Groups smaller than `min_group_n` (30 in our runs) are excluded, and `fairness_stats.py` adds a percentile bootstrap CI ($B = 1000$). Predictions come from the local inference backend running the imported model.

Mapping to O/S/D (heuristic). Let $\text{gap} = \max(|\text{DPD}|, |\text{EOD}|)$. Then $O = S = \text{clip}_{1..9}(\text{round}(10(1 - \text{gap})))$ and $D = 8$ (7 if a group is thin). For the label-flipped model V1, $\text{gap} = 0.418$ gives (6, 6, 8).

Interpretation and limits. DPD compares raw positive rates, so it also reports differences that come from real base-rate differences between groups. A model that is perfectly accurate on a dataset where one group contains more toxic comments will show a nonzero DPD. The EOD and F1-spread are less affected, but all three depend on a single sensitive attribute (here a binary “identity mentioned” flag).

4.4 Robustness

Implementation. `robustness_nlp.py` replaces up to `max_changes=3` alphanumeric characters per text with random lowercase letters or digits (seeded RNG), runs both the clean and the perturbed text through the model, and `robustness_eval.py` computes clean accuracy, robust accuracy and the paired accuracy drop with a bootstrap CI. A named risk `R-ROB-PERT` fires when the drop exceeds $\varepsilon = 0.05$ at 95% confidence. Gates reject the run if fewer than 100 samples were evaluated, if fewer than 95% of samples are label-compatible, or if clean accuracy is below 0.20.

Mapping to O/S/D. The heuristic engine sets $O = \text{scale}(\text{clean acc.})$, $S = \text{scale}(\text{robust acc.})$ and $D = \text{scale}(\min(\text{robust/clean}, 1))$. Two things follow, and both matter later. First, the band is dominated by raw accuracy, so a mediocre but stable model scores low. Second, a 4-point accuracy drop changes the ratio from 1.00 to about 0.95, and after rounding the band moves by at most one step.

Limits. Random character swaps are a weak attack. They test sensitivity to typos, not adversarial robustness; a gradient-guided attack [13] would be stronger. Models whose output collapses to one class are trivially stable under any perturbation, a failure mode we ran into when designing the robustness-flawed variant (Section 6.2.2).

4.5 Integrity

Implementation. `integrity_eval.py` (`tl-integrity-v1.0`) works on Hub metadata. It checks whether a card is present, whether the file list is available, whether the revision is pinned to a commit hash, whether a licence is declared, whether the file listing was recorded, and whether the card makes reproducibility claims (training data, seed, evaluation, hyperparameters). Named risks include `I-INT-REV-UNPINNED`, `I-INT-MANIFEST-MISSING`, `I-INT-LICENSE-UNDISCLOSED`, `I-INT-BYTES-DIVERGE` and `I-INT-LISTING-DRIFT`. Hash verification requires both a trusted reference hash and a local artifact hash; when either is missing the probe records the gates `G-INT-HASH-REF-MISSING` and `G-INT-HASH-LOCAL-MISSING` and states that cryptographic identity was *not* verified.

Limits. The probe documents identity and disclosure. It does not detect tampered weights unless a trusted reference hash is supplied, and in none of our runs was one supplied. The code’s own limitation text says so: integrity risks “measure disclosure and identity recording, not tampering or legal compliance”.

4.6 Explainability

Implementation. In TrustLens, explainability means transparency of documentation. `explainability_card.py` searches the card for five required sections (intended use, limitations, training data, evaluation, ethical considerations; bonus sections are tracked separately), and the coverage ratio is the number of required sections present divided by five. `explainability_eval.py` also flags contradictions such as `open_claim_vs_restrictive_license`

and `no_limitations_but_production_claim`. Template placeholders such as “[More Information Needed]” are rejected by `card_markdown.nontrivial()`.

Limits. This is not SHAP or LIME and says nothing about whether a human can understand *why* the model made a prediction. It measures whether the author wrote the sections, and the heuristic reads headings, so a card with the right headings and empty content can pass the probe (the LLM step in `11m_v1` is meant to catch this).

4.7 Safety

Implementation. `safety_card.py` (`tl-safety-v1.0`) looks for four disclosure items: misuse risks, privacy, security warnings and data disclosure, and flags high-impact deployment claims (for example “production ready for any use”) made without full disclosure.

Limits. There is no behavioural safety test set in the repository: nothing here feeds unsafe prompts to the model or checks how often it misses harmful content. The Safety score therefore describes the governance paperwork, not the model’s behaviour. This limitation drives the main negative result of Chapter 7.

4.8 O/S/D engines

Deterministic Abstains: all O, S, D are None, FRIES is withheld. Default, because the project has no validated metric-to-O/S/D mapping.

Heuristic The band rules above. For cards: $O = S = D = \text{scale}(\text{coverage})$, with $S-2$ and $D-1$ when high-impact claims have gaps; an empty card gets the fixed band (2, 2, 3). Integrity: $O = S = \text{scale}(\text{pass rate})$, $D = 8$.

LLM-assisted (`11m_v1`) One batched prompt asks a hosted model (Gemini first, then Groq, then NVIDIA NIM, in that order of preference) to give O, S, D from 1 to 9 for Integrity, Explainability and Safety, using the quoted card text and the probe evidence, with a 3–5 sentence rationale. If every provider fails, the heuristic values are kept and tagged `heuristic_fallback`. All three are labelled “proposed, not ground truth” in the report.

5 Implementation Details

5.1 Technology stack

Table 5.1 lists the technologies that the code actually imports. Libraries that only appear in dependency files are not listed.

Table 5.1: Technology stack and role of each component.

Technology	Role in TrustLens
Python ≥ 3.11	Backend, worker, probes, experiment scripts.
FastAPI, Pydantic v2	REST API under /v1; Pydantic models define the evaluation contract, probe output and report schemas.
SQLAlchemy 2, Alembic, PostgreSQL	Persistence; fourteen migrations track the schema (e.g. dataset content, draft state, methodology version).
Celery, Redis	Asynchronous execution of the evaluation pipeline in a worker process.
Hugging transformers [18], PyTorch	Local inference in LocalHFBackend; AutoConfig for cheap model inspection.
huggingface_hub	Metadata import (card, tags, licence, file list).
google-genai, httpx	LLM calls to Gemini, Groq and NVIDIA NIM for the llm_v1 engine.
Jinja2, WeasyPrint	HTML report template rendered to PDF.
React 19, Vite, TypeScript, React Router	Dashboard: model import, draft wizard, evaluation timeline, review and report pages.
Vitest, pytest, ruff	Frontend tests, backend tests, linting (also run in GitHub Actions).
Docker Compose	Five services: postgres, redis, api, worker, frontend; separate CPU-only and GPU compose files.

The GPU-related choices follow from the reference machine described in the specification: an RTX 4060 Laptop GPU with 8 GB of VRAM. Our experiment logs confirm CUDA execution with float32 weights and batch size 8 (`execution_metadata`).

5.2 Important modules

Table 5.2: Key source files and responsibilities.

File	Purpose, inputs and outputs
<code>inference/local_hf.py</code>	Loads a sequence-classification model and tokenizer, batches texts, applies softmax, returns predictions plus device metadata. Raises typed errors on load or inference failure.
<code>inference/device.py</code>	Chooses CUDA or CPU and records the reason (<code>device_reason</code>) and any fallback.
<code>probes/runner.py</code>	Runs probes in order, validates each output (dimension match, confidence in $[0, 1]$, evidence present), persists <code>probe_results</code> and refines confidence.
<code>probes/fairness.py</code> , <code>fairness_metrics.py</code> , <code>fairness_stats.py</code>	Subgroup predictions, DPD/EOD/F1 spread, bootstrap CI.
<code>probes/robustness_nlp.py</code> , <code>robustness_eval.py</code>	Char-swap attack and accuracy-drop evaluation with gates.
<code>probes/integrity_eval.py</code> , <code>explainability_eval.py</code> , <code>safety_eval.py</code>	Pure functions over Hub metadata and card text.
<code>confidence/engine.py</code>	Evidence-strength score: geometric mean of data quality, probe reliability and evidence completeness. Marked uncalibrated.
<code>osd/agent.py</code> , <code>hybrid.py</code> , <code>llm_client.py</code>	Heuristic bands; LLM prompt, call, parse, fallback.
<code>scoring/fries.py</code>	Cube-root risk score, veto, aspect mean, weighted total.
<code>reports/builder.py</code>	Builds the canonical <code>report_v1</code> JSON; PDF is a projection of it. Append-only versions.

Evidence and reproducibility features. Each probe stores its raw output as a JSON artifact with a SHA-256 hash; evaluations are stamped with an immutable `methodology_version`; model revisions are frozen at draft time and the loaded snapshot is verified against it (`model_snapshot_check.py`); and probes take an explicit seed. Our determinism check is in Section 7.4.

Documentation discrepancy. The project README still lists JWT roles (researcher, reviewer, admin) among the working features, and the setup script prints a `seed_users` step. The code does not match: migration `008_remove_auth_and_ownership` removed authentication and ownership, and there is no `seed_users` module under `backend/app/scripts`. We follow the code, and we omit that step from the commands below.

Error handling. Probe outputs that violate the contract raise `ProbeError` and fail the evaluation rather than being stored. A probe that cannot run records a status (`INSUFFICIENT_EVIDENCE`, `SKIPPED`, `NOT_APPLICABLE`, `FAILED` or `PROXY`) instead of a number. The API returns a conflict

(HTTP 409) for a report on an evaluation whose FRIES score was withheld, rather than pretending the report is merely not ready.

5.3 Installation and execution

The following commands are taken from the repository (`docs/TRUSTLENS_LOCAL_DEVELOPMENT.md`, `scripts/`, `Makefile`). Prerequisites: Python 3.11+, Node 18+, Docker Desktop with Compose v2; an NVIDIA GPU is optional but used when present.

```
# one-time backend setup (Windows PowerShell)
.\scripts\setup-local-dev.ps1
.\.venv\Scripts\Activate.ps1
docker compose up -d postgres redis
cd backend; alembic upgrade head
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
# in other terminals: worker and frontend
.\scripts\dev-worker.ps1      # Celery worker
.\scripts\dev-frontend.ps1    # React dashboard (npm run dev)
# tests
cd backend; python -m pytest -q
```

To reproduce the forged-model experiment (docstrings of the three scripts under `backend/app/scripts`, run from `backend/`):

```
python -m app.scripts.prepare_flawed_suite_data      # builds eval_set + per-variant train
    slices
python -m app.scripts.train_flawed_suite --all      # fine-tunes variants 1,2,3,5,6
python -m app.scripts.run_flawed_suite_eval        # evaluates all 7 models through the
    API
```

The last script needs the Compose stack running and the local model directory bind-mounted into the worker at `/models/flawed_model_suite`. Output is written to `results/flawed_model_suite/`. A user normally interacts through the dashboard: import a model, create an evaluation draft, attach a dataset and label mapping, confirm, and read the report. In the report, the FRIES score is shown next to the evidence, the O/S/D ratings with their rationale, the evidence-strength confidence, and a disclosure that states the mode, the engine and whether a human reviewed the numbers.

6 Experimental Design

6.1 Question and hypothesis

The research question for the experiment is RQ2 from Chapter 1: *can TrustLens distinguish a reference model from models that we deliberately made worse?*

We state the hypothesis in a form that can fail:

H1. If the FRIES probes capture trust-relevant properties, then a forged model should obtain a *lower score on the FRIES dimension in which its flaw was injected* than the reference model does, and a model carrying flaws in several dimensions should obtain the lowest overall score.

We also record a weaker, secondary hypothesis, **H2**: the overall FRIES score of each forged model is lower than that of the reference. We expected H2 to be harder to satisfy, because an overall score averages over four dimensions that were left untouched. We wrote H1 and H2 before looking at the second run, but we had already seen the first run, so neither is a fully pre-registered prediction.

6.2 Models

6.2.1 Model A: reference model

Table 6.1: Reference model.

Model	<code>unitary/toxic-bert</code> (Hugging Face Hub), the Detoxify BERT-based multi-label toxic comment classifier
Revision	4d6c22e74ba2fdd26bc4f7238f50766b045a0d94 (pinned in the evaluation contract)
Architecture	BERT sequence classifier with six output labels: <code>toxic</code> , <code>severe_toxic</code> , <code>obscene</code> , <code>threat</code> , <code>insult</code> , <code>identity_hate</code>
Parameter count	[INFORMATION REQUIRED: read from model files; not recorded in the result JSON]
Training data, licence	As stated on the Hub card; our probes record that the card lacks explicit intended-use and training-data sections (coverage 0.6)
Conversion for evaluation	TrustLens decides with argmax over a softmax, which cannot express “toxic versus not toxic” on a six-way sigmoid head. We therefore built <code>reference_toxicbert_2label</code> (<code>convert_reference_2label.py</code>): encoder weights unchanged; new head with $\text{logit}_0 = 0$ and logit_1 equal to the original <code>toxic</code> logit, so $P_1 = \sigma(z_{\text{toxic}})$ and argmax picks “toxic” exactly when the original output exceeds 0.5. A check on four sentences gave a maximum probability difference of 9.5×10^{-4} , so predictions very close to the boundary could differ from the original model.
Selection reason	Widely used, public, same task as our variants. <i>It is not a certified trustworthy model</i> ; we call it the reference model because it is an established public release, not because we have proof of its quality.

6.2.2 Model B: forged variants

We could not obtain a real malicious model, so we built six. All start from `bert-base-uncased` [17] and are fine-tuned for binary toxicity classification on a sample of Civil Comments [11]; each variant’s flaw was chosen to target one FRIES dimension, and the flip logic is recorded in a data manifest next to the weights. Table 6.2 summarises them.

Table 6.2: The six forged variants. All use `bert-base-uncased`, seed 42, max sequence length 128. “Rows” is the training set size.

Variant	Target	Injected flaw	Training
V1	Fairness	40% of non-toxic comments that mention an identity (any annotator flagged an identity attack) had their label flipped to toxic: 249 of 8 000 rows. Card is complete and honest about the flaw.	3 epochs, lr 2×10^{-5} , wd 0.01, 8 000 rows
V2	Robustness	Class-balanced narrow training set (all 628 toxic rows plus 628 non-toxic), 4 epochs, lr 5×10^{-5} , no weight decay, intended to produce a brittle model. Redesignated after a first design collapsed to a majority-class predictor.	4 epochs, 1 256 rows
V3	Explainability	Weights trained on clean labels; the model card is the untouched auto-generated template (“[More Information Needed]” everywhere).	3 epochs, 8 000 rows
V4	Integrity	<i>Weights are an exact copy of V1.</i> The card claims “production ready”, “no known biases”, “no material risks”, declares licence other while claiming unrestricted commercial use, and omits the limitations.	none (copy of V1)
V5	Safety	50% of rows with any severe-toxicity annotation relabelled as non-toxic (244 of 8 000), so the model under-flags severe toxicity. Card is complete and honest.	3 epochs, 8 000 rows
V6	Compound	V1 flip and V5 relabelling applied in sequence; template card with over-claiming sentence added; licence other .	3 epochs, 8 000 rows

Each fine-tuning took about 160 seconds on the RTX 4060 laptop GPU (V2, with fewer rows, is not timed in its manifest). Each resulting checkpoint is a float32 BERT-base model of 437.96 MB (about 109.5 million parameters including the classification head).

What the forged models are, and are not. They are experimental constructs for probing the tool. They do not represent real-world malicious models: nobody planted a backdoor, nothing adapts to the evaluator, and several flaws sit in the model card, which an attacker could trivially fix. Three of the six variants (V3, V4, V6) are essentially *documentation* flaws wrapped around weights that behave like an ordinary fine-tuned BERT, and V4 is behaviourally identical to V1. This matters for what a “detection” means, and we come back to it in Chapter 8.

A design lesson. Our first V2 used 600 rows and 12 epochs to overfit. Instead it collapsed to a majority-class predictor that predicted positive for only 0.6–8.4% of inputs, which is more stable under perturbation than the reference, not less (accuracy drop 0.3–0.9 points). The manifest records this diagnosis. It taught us that a robustness probe built on accuracy drop cannot see a constant predictor as a problem, and the same issue appears again in the reference model (Section 7.3).

6.3 Variables and controls

Table 6.3: Experimental variables.

Independent variable	Which model is evaluated (reference or one of V1–V6): weights, model card and licence field.
Dependent variables	Per-probe measured evidence (DPD, EOD, F1 spread, clean and robust accuracy, card coverage, integrity risks); per-dimension scores; overall FRIES score.
Controlled	Evaluation dataset (3 000 rows of the Civil Comments <code>test</code> split, seed 42, 20% positive rate, identical for all seven models); sensitive attribute (binary <code>identity_ref</code> flag); label mapping; <code>min_group_n</code> = 30; probe seed 42; assessment engine <code>llm_v1</code> ; mode <code>AI_AUTONOMOUS</code> ; hardware (RTX 4060 Laptop, CUDA, float32, batch 8); software stack.
Not controlled	The LLM that rates Integrity, Explainability and Safety (provider and sampling are not pinned or logged in the result file); the Hub-versus-local difference in model identity (Section 8.3); the architecture and label head of the reference model, which differ from the variants’.

Sensitive attribute. `identity_ref` is 1 when the Civil Comments row has `identity_attack` > 0.0001; group sizes in the evaluation set are 2 461 (value 0) and 539 (value 1). The same threshold is used when the V1 and V6 flip was injected, so the fairness probe evaluates the very attribute that the flaw targets. We note this as a favourable design for detection.

Runs. The full set of seven evaluations was executed twice. Run 1 was created between 20:49 and 20:57 UTC on 14 September 2026, with the redesigned V2 evaluated at 21:23. Run 2 was created between 23:57 UTC that day and 00:06 UTC on 15 September. The two runs are *not* exact replicates: Run 2 records an extra integrity gate (`G-INT-LISTING-UNVERIFIED`) that Run 1 does not, so the integrity probe changed in between, and the LLM that rates three dimensions is stochastic. We therefore treat them as two independent measurements of a slightly evolving tool. *Run 3* is a single evaluation of the converted reference model (`eval_results_v3`), made after we found the label-mapping problem described in Section 7.3. It uses the same dataset, mapping, mode, engine and hardware as Runs 1 and 2. The original Hub import of `toxic-bert` is called the *original reference* below; the converted copy is the *corrected reference*. Because the corrected reference is a local directory, the integrity probe treats it like the variants (no pinned revision, no file manifest), which makes its Integrity score comparable to theirs but not to the original reference’s. Run 1 also contains an earlier V2 evaluation of the first (collapsed) V2 design, which we exclude from the tables; it remains in the repository. All figures and tables below are computed from the saved JSON files by `report/make_figures.py`.

6.4 Outcome measures

For each model we report the five dimension scores and the overall FRIES score as produced by the tool, and the underlying evidence. For H1 we compare the *targeted* dimension of each

variant with the same dimension of the reference model. For H2 we compare overall scores. Because the sample is seven models, we use counts and plain differences, and we do not report significance tests.

7 Results

All numbers in this chapter come from the saved evaluation JSON files in `results/flagged_model_suite/eval_` (Run 1) and `eval_results_v2` (Run 2). “G” in tables is the *original* reference (Hub import, six-label head, wrong label mapping); “R*” is the *corrected* reference (Run 3), the one we use for hypothesis testing. Each evaluation reached status `FINALIZED` with five evaluated probes and no failures.

7.1 Overall and per-dimension scores

Table 7.1: FRIES scores, Run 2. The flawed dimension of each variant is in bold. Scores are on a 0–10 scale; higher is better.

Model	Fair.	Rob.	Integ.	Expl.	Safety	FRIES
G: toxic-bert (reference)	8.65	8.32	5.01	3.63	1.59	5.44
V1 fairness	6.60	9.00	3.91	9.00	3.91	6.49
V2 robustness	5.85	8.32	4.64	8.00	4.38	6.24
V3 explainability	7.32	9.00	1.82	1.82	1.82	4.35
V4 integrity	6.60	9.00	3.30	2.62	2.29	4.76
V5 safety	8.00	9.00	3.63	9.00	3.98	6.72
V6 compound (F+S, card)	6.60	9.00	2.29	1.26	1.26	4.08

Table 7.2: FRIES scores, Run 1 (V2 is the redesigned model used in both runs).

Model	Fair.	Rob.	Integ.	Expl.	Safety	FRIES
G: toxic-bert (reference)	8.65	8.32	7.32	6.00	3.63	6.79
V1 fairness	6.60	9.00	4.31	8.32	4.58	6.56
V2 robustness	5.85	8.32	4.00	8.65	2.88	5.94
V3 explainability	7.32	9.00	1.00	1.00	1.00	3.86
V4 integrity	6.60	9.00	2.00	2.29	1.59	4.30
V5 safety	8.00	9.00	4.82	8.65	2.88	6.67
V6 compound	6.60	9.00	2.62	1.59	1.59	4.28

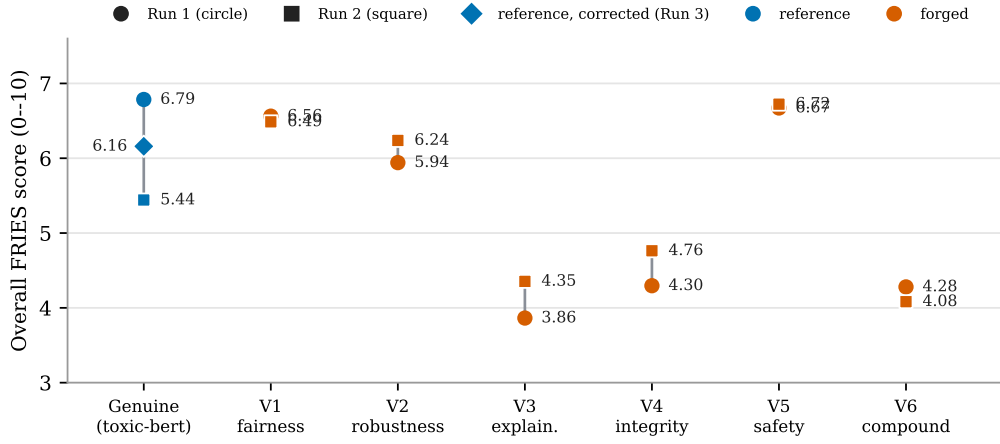


Figure 7.1: Overall FRIES score of every model in both runs. A vertical line joins the two runs of one model. The original reference moves the most between runs (1.34 points); the diamond is the corrected reference (Run 3).

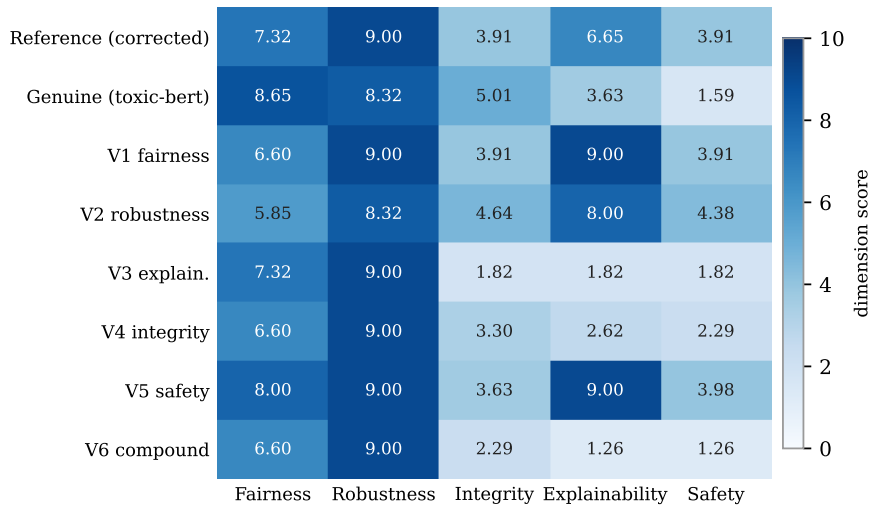


Figure 7.2: Dimension scores in Run 2; the first row is the corrected reference from Run 3. The columns that are uniformly dark for the variants with filled-in cards (Explainability, Safety) and the light rows for V3, V4 and V6 show where the score differences come from.

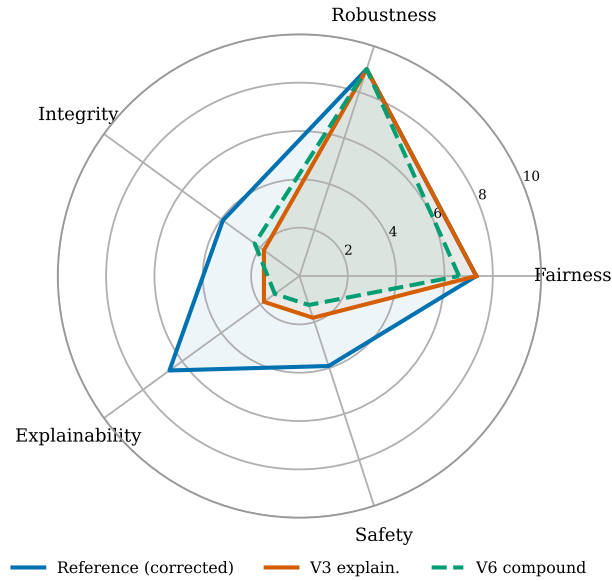


Figure 7.3: Run 2 FRIES profiles of V3 (blank card) and V6 (compound) against the corrected reference R^* (Run 3). R^* is weakest on Integrity and Safety, but V3 and V6 fall far below it on Explainability and Safety.

7.2 Measured evidence behind the scores

Table 7.3 lists what the probes actually measured in Run 2; the corrected reference follows in Table 7.4. Fairness and Robustness were measured by running the model on the 3000 rows; the other three columns come from the card and metadata.

Table 7.3: Measured evidence, Run 2. DPD with 95% bootstrap CI; Drop is the char-swap accuracy drop in percentage points; Card cov. and Safety cov. are the fractions of required sections or checks found.

	DPD [CI]	EOD	F1 spr.	Clean acc.	Drop	Card cov.	Safety cov.
G	0.000 [0.000, 0.000]	0.000	0.000	0.799	−0.07	0.60	0.00
V1	0.418 [0.376, 0.464]	0.164	0.123	0.897	1.47	1.00	0.25
V2	0.526 [0.487, 0.561]	0.370	0.294	0.809	4.10	1.00	0.25
V3	0.319 [0.279, 0.363]	0.065	0.074	0.894	1.17	0.60	0.25
V4	0.418 [0.376, 0.464]	0.164	0.123	0.897	1.47	0.60	0.25
V5	0.233 [0.197, 0.275]	0.035	0.033	0.876	1.30	1.00	0.25
V6	0.368 [0.327, 0.413]	0.167	0.161	0.885	1.30	0.00	0.00

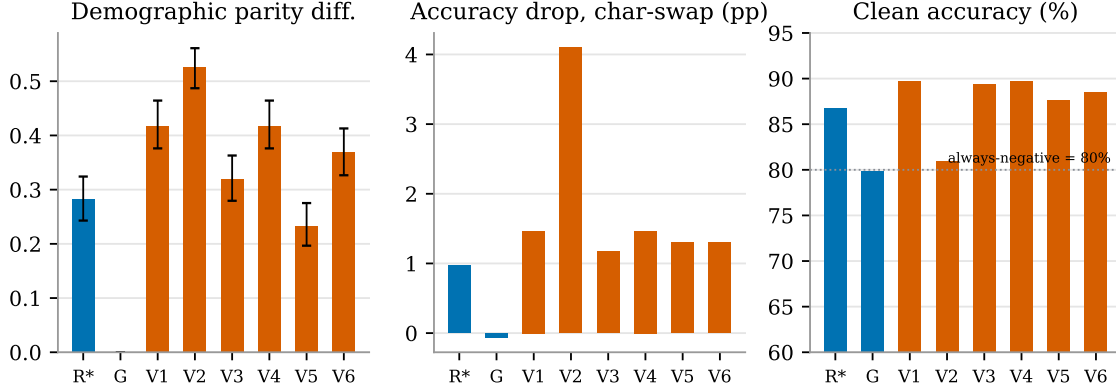


Figure 7.4: Measured fairness and robustness evidence, Run 2. R* is the corrected reference (Run 3) and G the original one with the wrong label mapping. Left: demographic parity difference with bootstrap CI. Middle: accuracy drop under character swaps. Right: clean accuracy; the dotted line is the accuracy of a model that always answers “not toxic” on this sample.

Three observations from Table 7.3 and Figure 7.4 deserve attention.

- **V1 and V4 are identical in behaviour**, as they should be, since V4 reuses V1’s weights. Every Fairness and Robustness metric, CI and O/S/D triple matches exactly. This is a useful sanity check that the behavioural probes are deterministic for fixed weights, data and seed.
- **The fairness probe separated the flipped model from the clean model.** V1 has a DPD of 0.42 [0.38, 0.46]; V3, trained on the same data with no flip, has 0.32 [0.28, 0.36]. These intervals do not overlap. But V3’s DPD is far from zero as well, and V5 and V6 show intermediate values, which is expected if part of the gap comes from the data and not from the injected flaw.
- **The robustness flaw produced the largest accuracy drop (4.1 points), but the probe did not raise a risk.** The R-ROB-PERT trigger needs a drop of at least 5 points at 95% confidence, and V2 sits below it. V2 also has the largest fairness gap (DPD 0.53), a side effect of its narrow training set that we did not design.

7.3 The original reference was a constant predictor, and the corrected rerun

7.3.1 Diagnosis

The original reference model’s demographic parity, equalized-odds and F1-spread are exactly zero, its clean accuracy is 0.7987, and the saved per-group statistics show a positive-prediction rate of 0.0 in *both* identity groups: it never predicted “toxic” on this sample. Since 20% of the sample is toxic, always answering “not toxic” would score 0.80, and the model scored 0.799. The cause is the label mapping. `unitary/toxic-bert` has six outputs, with `toxic` at index 0 and `severe_toxic` at index 1, and our contract mapped dataset value 1 to index 1, so the probe checked whether `severe_toxic` was the winning output. Its Fair-

ness (8.65) and Robustness (8.32) scores in Tables 7.1 and 7.2 therefore say nothing about a working toxicity classifier: a constant predictor is trivially fair and trivially robust, and TrustLens raised no warning about it.

7.3.2 Correction

The tool offers no mapping that expresses “toxic versus not toxic” on this head: with `argmax` over six outputs a benign comment still selects some label, and no index stands for “none”. We therefore converted the model to a two-label head with identical decision behaviour (Section 6.2.2 and Table 7.4) and evaluated it through the unchanged pipeline, with the standard mapping (0→0, 1→1) and `positive_label_index=1` (Run 3).

Table 7.4: Corrected reference model (Run 3) against the original reference (Run 2).

	Original (G)	Corrected (R*)
Clean accuracy	0.799	0.868
Positive-prediction rate, group 0 / group 1	0.000 / 0.000	0.051 / 0.334
DPD [95% CI]	0.000 [0.000, 0.000]	0.283 [0.243, 0.324]
EOD / F1 spread	0.000 / 0.000	0.174 / 0.203
Char-swap accuracy drop (pp)	−0.07	0.97
Fairness / Robustness score	8.65 / 8.32	7.32 / 9.00
Integrity / Explainability / Safety	5.01 / 3.63 / 1.59	3.91 / 6.65 / 3.91
Overall FRIES	5.44	6.16

The corrected reference is a working classifier (86.8% accuracy) and it, too, shows a demographic-parity gap of 0.28, because comments that mention identities are toxic more often in this dataset and the model reflects that. This is evidence that part of any DPD in this experiment is a base-rate effect and not a flaw, and it sets the fair comparison for V1: the label-flipped model’s gap of 0.42 [0.38, 0.46] is clearly above the corrected reference’s 0.28 [0.24, 0.32], while the unflipped V3 (0.32 [0.28, 0.36]) overlaps with it. The Integrity score of 3.91 is low for the same reason as the variants’ scores: two named risks (I-INT-REV-UNPINNED, I-INT-MANIFEST-MISSING) fire for any local directory. Its Explainability score (6.65) comes from a card with three of five required sections.

7.4 Run-to-run variation

Figure 7.5 compares the two runs on the three dimensions that the LLM rates. Fairness and Robustness scores are identical across runs for every model, because they are computed from deterministic probe metrics by fixed rules. The differences are therefore entirely in Integrity, Explainability and Safety.

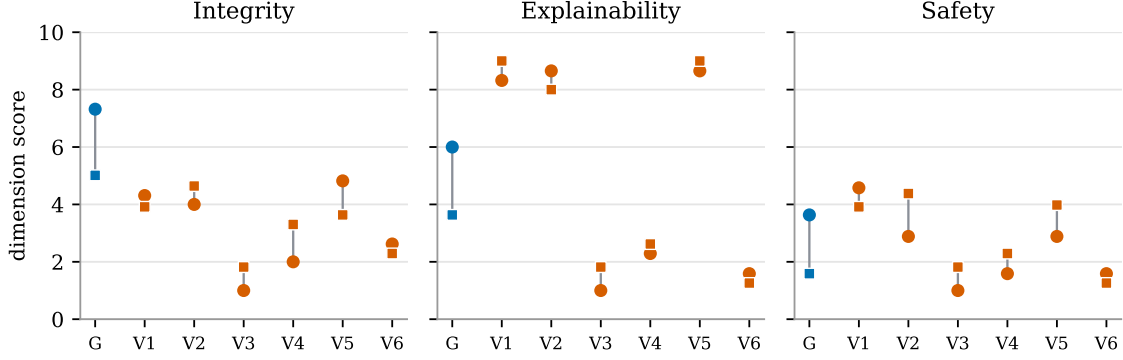


Figure 7.5: Dimension scores in Run 1 (circles) and Run 2 (squares) for the three LLM-rated dimensions. Blue is the reference model; orange are the forged variants.

The overall score changes by 0.05 to 0.49 points for the forged models and by 1.34 points for the reference model (Table 7.5). The reference model’s Integrity score dropped from 7.32 to 5.01, its Explainability score from 6.00 to 3.63 and its Safety score from 3.63 to 1.59, even though its card and metadata are the same in both runs. Spearman rank correlation of the seven overall scores between the two runs is 0.75. The ordering of the lowest three (V3, V4, V6 below the rest) is stable; the position of the reference model is not (first in Run 1, fourth in Run 2).

Table 7.5: Overall FRIES score by run, and the difference.

Model	Run 1	Run 2	Change	Rank (1 / 2)
G	6.785	5.442	−1.343	1 / 4
V1	6.562	6.487	−0.076	3 / 2
V2	5.941	6.238	+0.296	4 / 3
V3	3.864	4.354	+0.490	7 / 6
V4	4.296	4.763	+0.467	5 / 5
V5	6.672	6.723	+0.051	2 / 1
V6	4.280	4.083	−0.197	6 / 7

Determinism of the non-LLM path. Within each run, the Fairness and Robustness metrics of V1 and V4 coincide exactly, and across runs those two dimensions never changed. This supports the claim that the probe pipeline itself is deterministic given a pinned seed.

7.5 Testing hypotheses H1 and H2

We compare each variant (Run 2) with the *corrected* reference (Run 3), and also report the comparison with the original reference for completeness.

Table 7.6: Hypothesis check against the corrected reference R^* . “Targeted dimension” compares the variant’s score on the dimension where its flaw was injected with R^* ’s score on that dimension. Overall scores: R^* 6.16.

Variant	Dimension	R^*	Variant (Run 2)	H1 holds?	Overall below R^* ?
V1	Fairness	7.32	6.60	yes	no (6.49)
V2	Robustness	9.00	8.32	yes	no (6.24)
V3	Explainability	6.65	1.82	yes	yes (4.35)
V4	Integrity	3.91	3.30	yes (small)	yes (4.76)
V5	Safety	3.91	3.98	no	no (6.72)
V6	Safety (and Expl.)	3.91	1.26	yes	yes (4.08)

For V6 we use Safety, one of its two injected flaws; its Explainability score (1.26) is also far below R^* ’s 6.65.

H1 held for 5 of 6 variants against the corrected reference. The Integrity margin for V4 is 0.6 points, smaller than the run-to-run spread we observed for LLM-rated dimensions (Table 7.5), so we would not call it a robust detection. The Robustness margin for V2 (8.32 against 9.00) is one band step, produced by a 3-point larger accuracy drop. V5 fails: its Safety score is 0.07 higher than R^* ’s. **H2** held for 3 of 6 (V3, V4, V6). The mean overall score of the six forged models is 5.44 against 6.16 for R^* . Against the *original* reference the figures are 6.79 and 5.44 (Runs 1 and 2), but we regard those comparisons as invalid for the Fairness and Robustness dimensions.

7.6 Cost

Evaluating the corrected reference (Run 3) took the same order of time. Fine-tuning one variant took about 160 s on the RTX 4060 laptop GPU. The seven evaluations of a run were created roughly 80–135 seconds apart (from the `created_at` timestamps); this is an upper bound on one evaluation, including model loading, two inference passes over 3 000 texts for the fairness and robustness probes, the perturbed pass, the LLM call and database writes. The result files do not record peak GPU memory or per-stage timings, so we do not report them. [INFORMATION REQUIRED: measure peak VRAM/RAM and per-stage time with a profiler and add here]

8 Discussion

8.1 Does TrustLens identify the trustworthiness of a model?

The honest answer from our data is: *partly, and mostly for the part of trust that lives in documentation and metadata.*

8.1.1 Evidence that supports the claim

- **Documentation and integrity flaws were detected consistently.** V3, V4 and V6 had the three lowest overall scores in both runs, between 3.86 and 4.76, against 6.16 for the corrected reference. Their scores on the flawed dimension were below the reference’s in every run.
- **The behavioural fairness probe responded to a behavioural flaw.** The label-flipped model V1 produced a DPD of 0.42 with a confidence interval that excludes both the 0.32 of the unflipped V3 and the 0.28 of the corrected reference, and the Fairness score fell to 6.60 against 7.32. The probe is deterministic, and results for V1 and V4 (same weights) matched exactly.
- **The compound model scored lowest overall in Run 2 and near the bottom in Run 1,** which matches the intuition that two flaws should score worse than one.
- **Evidence is traceable.** For every number above there is a stored probe output with a hash, a rationale and a status. Even where we disagree with the score, we can see why it came out as it did. This is how we found the constant-predictor problem.

8.1.2 Evidence against, and limits on the claim

- **The safety flaw was not detected, and the overall score favoured three flawed models.** V5 under-flags severe toxicity, a genuine behavioural weakness, but the Safety probe never looks at behaviour. V5’s card is complete and honest about the flaw, so the card-based probes rate it at least as well as the reference model, whose card is thin (Safety 3.98 against 3.91). In a sense the tool did what it was designed to do (reward disclosure) and that is exactly the problem: an honest card on a harmful model is rated above a sparse card on a possibly good model.
- **The robustness flaw was only weakly visible.** V2 lost 4.1 points of accuracy under character swaps, four times the next variant and four times the corrected reference’s 0.97, but the Robustness score moved only from 9.00 to 8.32 because the O/S/D band is dominated by accuracy, and the R-ROB-PERT trigger needs a 5-point drop.
- **Three of five dimensions depend on an LLM and moved between runs.** The original reference model’s overall score changed by 1.34 points between two evaluations

of identical inputs, enough to change its rank from first to fourth. We evaluated the corrected reference only once, so its own spread is unknown.

- **Our first reference evaluation was invalid, and we had to correct it.** The original Hub import behaved as a constant predictor, which gave it a perfect-looking Fairness and Robustness score. TrustLens did not warn about this. After converting the model to an equivalent two-label head, the reference scored Fairness 7.32 and Robustness 9.00 with a genuine demographic-parity gap of 0.28. The corrected comparison is the one we rely on, but it is a single evaluation, and the conversion itself (maximum probability difference 9.5×10^{-4} on a four-sentence check) is a modelling step we introduced.
- **Most of the signal comes from documents we wrote.** V3, V4 and V6 differ from clean models in their cards and licence fields. A forger who knows TrustLens could fix them in minutes.
- **The experiment is small.** Seven models, one task, one dataset, one forger (us), two runs. We report counts and differences, not significance tests, because the sample cannot support them.

8.1.3 Is it trustworthiness or an audit profile?

We think the second description is more accurate. TrustLens produces an *audit profile*: a structured record of what was measured, what was disclosed and what is missing, with a number attached that summarises a set of heuristic rules. The number is useful for sorting models that differ a lot in documentation quality, and it is not a measurement of trustworthiness in a sense that could be validated against ground truth, because no ground truth exists for the models in this experiment other than the flaws we ourselves injected. The project README makes the same claim about the score; we agree with it and would add that the evidence from Chapter 7 supports the caution.

8.2 Questions we asked of our own design

Is FRIES sufficient? No. It omits privacy, calibration, provenance beyond metadata, and environmental or legal aspects discussed in the surveys [5]. TrustLens implements five of many possible dimensions.

Are the thresholds justified? Not empirically. The README labels every metric-to-O/S/D band “PROPOSED / REQUIRES VALIDATION”. $\varepsilon = 0.05$, the 95% intervals, the min-group size of 30 and the band formulas are design choices, not calibrated values.

Could the difference come from something unrelated to trust? Yes. Variants differ in training data size, class balance and learning rate as well as in the injected flaw, and the reference differs from them in architecture head and training data. Also, the integrity probe penalises the local variants for lacking a pinned revision and a manifest (I-INT-REV-UNPINNED, I-INT-MANIFEST-MISSING); those risks fire for *every* variant because they are local directories and not Hub repositories, so part of the Integrity gap between the reference model and the variants reflects where the file lives and not what it contains.

Does the overall score have statistical meaning? No. It is a weighted mean of cube roots of integer ratings; the ratings are ordinal and partly LLM-generated.

Could a malicious model evade the probes? Easily. See Section 8.4.

8.3 Threats to validity

Internal validity. Do the scores measure what we claim? For Fairness and Robustness, the metrics are well defined, but the reference model’s mapping error shows that a valid-looking number can measure the wrong thing. Run 1 and Run 2 used slightly different integrity code, so we cannot attribute their differences to the LLM alone. We did not log the LLM provider and temperature per evaluation, and we could not check which provider answered. DPD conflates a true base-rate difference with model unfairness; the groups in the Civil Comments sample have different toxicity rates, which explains why the unflipped V3 still shows DPD 0.32.

External validity. Seven models, one binary toxicity task, BERT-base variants, one dataset. Nothing here says how TrustLens behaves on image models, generative models, other languages, or models whose cards are well written but unfaithful. The robustness probe skips non-text models by design, and an earlier benchmark on six small models (documented in docs/RESULTS_SUMMARY.md) found the same dimension skipped for two of them.

Construct validity. FRIES as implemented measures: parity of predictions across one binary attribute; sensitivity to random typos; whether five card headings exist; whether metadata fields exist; whether four safety topics are mentioned. These are proxies. We do not show that they track the broader concept of trustworthiness, and the Safety and Explainability dimensions in particular are named after properties (preventing harm, explaining decisions) that the probes do not examine.

Reproducibility. The data preparation, training and evaluation scripts are in the repository and the seeds, flip positions and hyperparameters are stored in manifests, so another researcher can rebuild the variants and the evaluation set. They cannot reproduce the LLM-rated dimensions exactly, because hosted models change and the provider is not pinned. The Fairness and Robustness dimensions reproduce exactly given the same weights and data, as the V1–V4 identity and the unchanged scores across runs show. We did not rerun the experiment on a second machine.

8.4 Threat model

Table 8.1: Threats an auditing tool would like to catch, and what TrustLens does about them in its current form.

Threat	What TrustLens does	What it cannot do
Misleading model card (over-claims, empty sections)	Detects missing sections and some contradictions; LLM judges content quality.	Cannot verify that a well-written card is true.
Missing or inconsistent licence/metadata	Integrity risks for unpinned revision, missing manifest, undisclosed licence.	Cannot judge legal compliance.
Tampered weights	Compares SHA-256 to a trusted reference <i>if supplied</i> .	No reference was supplied in any experiment; hash-only checks cannot find backdoors in an unmodified-looking file whose hash is itself replaced.
Biased behaviour on a sensitive group	Measures DPD/EOD/F1 spread for one binary attribute with CIs.	Other attributes, intersections, and base-rate effects.
Fragility to input noise	Random character swaps.	Targeted or semantic attacks.
Unsafe outputs, missed harmful content	Checks that safety topics are disclosed.	Does not test behaviour at all.
Backdoors and poisoned behaviour [15]	Nothing specific.	Not in scope.
Evaluator-aware adversary	Nothing.	A forger who reads the repository can write a card that passes every card check and train a model that behaves well on public probes.

TrustLens therefore does not provide complete model security. It is closer to a structured checklist with some measurements, useful as one input to a human reviewer.

8.5 Lessons from building the forged models

Three things surprised us. First, getting a model to be *brittle* was harder than expected, because small or badly trained models collapse to the majority class and become stable. Second, the same effect meant that the reference model, through a mapping slip, also became stable and looked excellent. Third, the tool’s most reliable detections were of flaws that we had put in the card, which suggests that the first useful role of such a tool is to check disclosure and metadata consistency, and the second, still to be built, is to run richer behavioural tests.

9 Testing and Performance

9.1 Testing strategy

The backend has 85 files under `backend/tests` with 592 test functions by a simple count of `def test_`; the worker package and the frontend have their own tests (Vitest for components such as the report, review and draft pages). Continuous integration (`.github/workflows/ci.yml`) runs `ruff check` and `pytest`, including a job that skips tests marked `integration`, and installs and tests the frontend with `npm ci`. We did not generate a coverage report for this thesis, so we cannot give a coverage percentage. **[INFORMATION REQUIRED: run `pytest -cov` and insert line coverage]**

Table 9.1: Groups of automated tests (names from `backend/tests`).

Area	Examples and what they check
Scoring	<code>test_fries_scorer</code> : cube-root, veto, aspect mean, weights; compares against frozen vectors in <code>shared/scoring/fixtures/fries_test_vectors.json</code> (shared with the frontend implementation).
Probes	<code>test_fairness_contract_v2</code> , <code>test_fairness_stats</code> , <code>test_robustness_eval</code> , <code>test_robustness_nlp</code> , <code>test_integrity_probe</code> , <code>test_explainability_eval</code> , <code>test_safety_probe</code> , <code>test_probe_output_contract</code> .
Inference	<code>test_inference_backend</code> , <code>test_device_resolution</code> , <code>test_device_placement</code> , <code>test_model_snapshot_check</code> .
O/S/D engines	<code>test_deterministic_osd</code> , <code>test_osd_agent</code> , <code>test_hybrid_osd_agent</code> (fallback on LLM failure), <code>test_llm_client</code> .
Pipeline and API	<code>test_evaluation_lifecycle</code> , <code>test_e2e_draft_to_report</code> , <code>test_api_reports</code> , <code>test_report_concurrency</code> , <code>test_finalize_modes</code> , <code>test_human_review</code> .
Honesty and migration	<code>test_phase6_honesty</code> , <code>test_mode_disclosure_scoring_state</code> , <code>test_methodology_version_migration</code> , <code>test_migration_legacy_readback</code> .

Most probe tests use injected fake backends rather than loading real models, so they check logic and contracts, not behaviour on real checkpoints. The forged-model experiment of Chapter 7 is the only end-to-end test with real weights, and it found a problem that the unit tests could not (the label-mapping outcome). **[INFORMATION REQUIRED: record the pass/fail counts of the latest full test run]**

9.2 What the tests do not cover

There is no test that the heuristic bands produce sensible scores for a degenerate classifier, and no test comparing score ordering for a known good and a known bad model; the forged-model experiment is effectively the first such test. The original-reference problem shows the gap. We think the most valuable regression test to add is a smoke test that evaluates a constant predictor and expects a warning.

9.3 Performance and resource use

The system was run on the reference laptop: NVIDIA GeForce RTX 4060 Laptop GPU (8 GB VRAM) with CUDA, float32, batch size 8. For a BERT-base classifier (109.5M parameters, 438 MB on disk) this fits comfortably in VRAM. Measured costs are in the “Cost” section of Chapter 7: about 160 s to fine-tune a variant and roughly 80–135 s between consecutive finished evaluations. The README roadmap lists a resource checker and a benchmark-first runtime estimate as not yet implemented, so TrustLens does not currently predict whether a model will fit on the machine before loading it. Larger models would need that gate; for now, robustness may download and load a sequence-classification model at probe time with no memory check.

10 Conclusion and Future Work

We close by answering the three questions we set out in Chapter 1, one at a time, because they have different answers.

10.1 Q1. Did we successfully implement TrustLens?

Largely yes, within the scope we documented. The repository contains a working pipeline from Hub model import to local GPU inference, five probes, three O/S/D engines, the FRIES scorer verified against frozen vectors, human review, versioned reports, an evidence store and a dashboard, with 592 backend test functions. It does not contain several components that appear in our own target architecture: a resource checker, a benchmark-first runtime estimate, behavioural safety tests and SHAP/LIME explanations. Some probes are shallow, and the README itself lists ten known methodological gaps. The Explainability and Safety dimensions measure documentation only.

10.2 Q2. Did TrustLens detect measurable differences between the reference and forged models?

For some flaws, yes; for others, no. Against the corrected reference (overall 6.16), the score on the targeted dimension was lower for five of the six forged models (hypothesis H1): Fairness 6.60 against 7.32 with a clearly larger parity gap (0.42 against 0.28), Robustness 8.32 against 9.00, Explainability 1.82 against 6.65, Integrity 3.30 against 3.91 (a small margin), and Safety/Explainability 1.26 against 3.91 and 6.65 for the compound model. Models whose flaws lie in documentation and metadata (V3, V4, V6) also received clearly lower overall scores (4.08–4.76). On the other hand, the injected safety flaw (V5) was not detected (Safety 3.98 against 3.91), because the Safety probe does not examine behaviour; three of six forged models obtained a *higher* overall score than the reference (H2 held for 3 of 6), and the LLM-rated dimensions varied between runs (1.34 points for the original reference). Our first reference evaluation was invalid because of a label-mapping error that made the model a constant predictor; the figures above use the corrected rerun.

10.3 Q3. Does this prove that TrustLens can determine whether any arbitrary ML model is trustworthy?

No, and our data points the other way in several respects. Seven models, one task, flaws that we designed, a single evaluation of the corrected reference, a probe set that did not flag a constant-predictor model, and LLM-rated dimensions that vary between runs do not support a general claim. Even a perfect separation of our seven models would show only

that these probes can detect these flaws. TrustLens, in its current form, is best described as a *structured, partly automated audit profile*: it records evidence about measurable properties and disclosure, it makes assumptions explicit, and its overall score is a convenient summary, not a validated measure of trust.

10.4 Contributions, restated

Engineering: the implemented system and its evidence discipline. **Methodological:** the three-layer separation and explicit evidence statuses. **Empirical:** a forged-model suite with manifests and a candid comparison, including a documented negative result for the Safety dimension and the discovery of the constant-predictor trap. **Future research opportunity:** validating any such score against independent judgments of trust.

10.5 Future work

Each item follows from a limitation reported above.

1. **Support multi-label heads natively** (a decision rule on a chosen output, instead of our head conversion) and add a probe-level warning when a model predicts a single class for (almost) every input; repeat the corrected reference evaluation several times to measure its spread.
2. **Behavioural safety probes.** A test set of harmful and benign prompts or comments, so that V5-style flaws change the Safety score.
3. **Pin and log the LLM**, set temperature to zero where possible, repeat each evaluation several times and report the spread. Alternatively replace the LLM step with a rule-based or human-reviewed mapping and compare.
4. **Stronger robustness attacks** (word-level and gradient-guided perturbations) and a band that does not depend on raw accuracy.
5. **Fairness beyond one attribute**, and a metric that separates base-rate differences from model bias.
6. **More models and more forgers:** dozens of Hub models with human trust ratings from at least two independent reviewers, multiple forged models built by people who did not write the probes, and statistical tests on the results. This is the only route to validating the score.
7. **Provenance.** Supplying trusted reference hashes, verifying model-card claims against evidence, and scanning for known backdoor patterns.
8. **Resource gating and runtime estimates** before loading weights, as specified but not yet built.
9. **Calibration of the confidence engine** and of the O/S/D bands against empirical data.

None of these is implemented in the current repository.

Bibliography

- [1] J. Rutinowski *et al.*, “Benchmarking Trust: A Metric for Trustworthy Machine Learning,” in *Explainable Artificial Intelligence (xAI 2024)*, Springer, 2024. (FRIES trust score.)
- [2] M. Manzano, C. Ayala, and C. Gómez, “TrustML: A Python package for computing the trustworthiness of ML models,” *SoftwareX*, vol. 26, art. 101740, 2024.
- [3] R. Vashistha and A. Farahi, “I-trustworthy models: A framework for trustworthiness evaluation of probabilistic classifiers,” in *Proc. AISTATS*, 2025.
- [4] R. Vashistha and A. Farahi, “U-trustworthy models: Reliability, competence, and confidence in decision-making,” arXiv:2401.02062, 2024.
- [5] N. Kemmerzell, A. Schreiner, H. Khalid, M. Schalk, and L. Bordoli, “Towards a better understanding of evaluating trustworthiness in AI systems,” *ACM Computing Surveys*, 2025.
- [6] J. Mattioli *et al.*, “An overview of key trustworthiness attributes and KPIs for trusted ML-based systems engineering,” *AI and Ethics*, vol. 4, pp. 15–25, 2024, doi:10.1007/s43681-023-00394-2.
- [7] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” NIST AI 100-1, Jan. 2023.
- [8] M. Mitchell *et al.*, “Model cards for model reporting,” in *Proc. ACM FAT**, 2019, pp. 220–229.
- [9] T. Gebru *et al.*, “Datasheets for datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [10] M. Hardt, E. Price, and N. Srebro, “Equality of opportunity in supervised learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- [11] D. Borkan, L. Dixon, J. Sorensen, N. Thain, and L. Vasserman, “Nuanced metrics for measuring unintended bias with real data for text classification,” in *Companion Proc. The Web Conference (WWW)*, 2019, pp. 491–500.
- [12] N. Mehrabi, F. Morstatter, N. Saxena, K. Lerman, and A. Galstyan, “A survey on bias and fairness in machine learning,” *ACM Computing Surveys*, vol. 54, no. 6, 2021.
- [13] J. Ebrahimi, A. Rao, D. Lowd, and D. Dou, “HotFlip: White-box adversarial examples for text classification,” in *Proc. ACL*, 2018, pp. 31–36.
- [14] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, “Beyond accuracy: Behavioral testing of NLP models with CheckList,” in *Proc. ACL*, 2020, pp. 4902–4912.

- [15] T. Gu, B. Dolan-Gavitt, and S. Garg, “BadNets: Identifying vulnerabilities in the machine learning model supply chain,” arXiv:1708.06733, 2017.
- [16] W. Jiang *et al.*, “An empirical study of pre-trained model reuse in the Hugging Face deep learning model registry,” in *Proc. IEEE/ACM ICSE*, 2023.
- [17] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [18] T. Wolf *et al.*, “Transformers: State-of-the-art natural language processing,” in *Proc. EMNLP: System Demonstrations*, 2020, pp. 38–45.

A Repository Map

backend/	FastAPI /v1, probes, O/S/D engines, FRIES scorer, reports, tests
worker/	Celery consumer (same probe pipeline)
frontend/	Vite + React dashboard (TypeScript)
configs/	pinned datasets and earlier benchmark configuration
docs/	methodology notes, runbook, earlier results write-up
results/	experiment outputs; flawed_model_suite/ holds this report's data
shared/	frozen FRIES scoring test vectors
report/	this LaTeX report and make_figures.py

B Contents of the Forged-Model Suite

```
results/flawed_model_suite/  
  data/          eval_set.csv (3000 rows), per-variant training JSONL and manifests  
  cards/         model card (README.md) of each variant  
  models/        weights + train_manifest.json for variants 1-6 (+ the first V2 design)  
  eval_results/   Run 1 JSON per model + _summary.json  
  eval_results_v2/ Run 2 JSON per model + _summary.json  
  eval_results_v3/ Run 3: corrected (2-label) toxic-bert, one evaluation
```

The evaluation set manifest records dataset `google/civil_comments`, split `test`, seed 42, size 3000, positive rate 0.20, and thresholds 0.5 (toxicity) and 0.0001 (identity attack, severe toxicity).

C Corrected-Reference Run (Run 3)

```
python -m app.scripts.convert_reference_2label      # builds models/  
        reference_toxicbert_2label  
python -m app.scripts.run_reference_2label_eval     # one evaluation -> eval_results_v3/
```

Proposed O/S/D (Run 3): Fairness 7,7,8; Robustness 9,9,9; Integrity 3,4,5; Explainability 6,7,7; Safety 3,4,5; overall FRIES 6.16 (Fairness 7.32, Robustness 9.00, Integrity 3.91, Explainability 6.65, Safety 3.91).

D Raw Dimension Scores

Table D.1: Dimension scores for both runs as stored in `_summary.json` (rounded to two decimals).

Model	Run	F	R	I	E	S	FRIES
G	1	8.65	8.32	7.32	6.00	3.63	6.79
G	2	8.65	8.32	5.01	3.63	1.59	5.44
V1	1	6.60	9.00	4.31	8.32	4.58	6.56
V1	2	6.60	9.00	3.91	9.00	3.91	6.49
V2	1	5.85	8.32	4.00	8.65	2.88	5.94
V2	2	5.85	8.32	4.64	8.00	4.38	6.24
V3	1	7.32	9.00	1.00	1.00	1.00	3.86
V3	2	7.32	9.00	1.82	1.82	1.82	4.35
V4	1	6.60	9.00	2.00	2.29	1.59	4.30
V4	2	6.60	9.00	3.30	2.62	2.29	4.76
V5	1	8.00	9.00	4.82	8.65	2.88	6.67
V5	2	8.00	9.00	3.63	9.00	3.98	6.72
V6	1	6.60	9.00	2.62	1.59	1.59	4.28
V6	2	6.60	9.00	2.29	1.26	1.26	4.08

E Proposed O/S/D Triples

Table E.1: O/S/D triples (O,S,D) per dimension as proposed by the 11m_v1 engine, Run 2. Fairness and Robustness are heuristic bands; Integrity, Explainability and Safety are LLM-rated.

Model	F	R	I	E	S
G	9,9,8	8,8,9	7,6,3	3,4,4	1,2,2
V1	6,6,8	9,9,9	3,4,5	9,9,9	3,4,5
V2	5,5,8	8,8,9	4,5,5	8,8,8	3,4,7
V3	7,7,8	9,9,9	1,2,3	1,2,3	1,2,3
V4	6,6,8	9,9,9	3,3,4	2,3,3	2,2,3
V5	8,8,8	9,9,9	3,4,4	9,9,9	3,3,7
V6	6,6,8	9,9,9	2,2,3	1,1,2	1,1,2

F Regenerating the Report

```
cd report
python make_figures.py      # rebuilds figures/*.pdf from the result JSON files
tectonic main.tex           # or: pdflatex main.tex (run twice for the table of contents)
```

Institution, supervisor and academic-year fields are macros at the top of `main.tex`. Passages marked `[INFORMATION REQUIRED: ...]` in red need information that is not in the repository.